

CHAPTER-6 (Unit 1 - Part 2)

SOFTWARE METRICS

→ Metrics →

- It is unit of measurement of a software product or software related process.
- Metrics are aimed at getting empirical laws that relate program size to expected number of bugs, expected no. of tests required to find bugs, test technique effectiveness, resource requirement, release instant, reliability & quality requirements etc.

Definition of Metrics

- The continuous application of measurement based technique to the sw development process & its product to supply meaningful & timely management informations, together with the use of those techniques to improve that process & its product.

Area of Applications

- ① Prediction of quality level for software i.e. software reliability.
- ② Quantitative checks on sw design.
- ③ Provide management information i.e. productivity, quality & process effectiveness.
- ④ Estimation of cost & size.

Categories of Metrics

① Product Metrics → Describe the characteristics of the product such as size, complexity, design features, performance, efficiency, reliability, portability etc

② Process Metrics → Describe the effectiveness & quality of the process processes that produce the s/w product
e.g.

- effort required in the process
- time to produce the product
- Effectiveness of defect removal during development
- Number of defects found during testing
- Maturity of the process.

③ Project Metrics

Describe the project characteristics & execution
e.g. are

- no. of s/w developers
- staffing pattern over the life cycle of the s/w
- cost & schedule
- productivity

TOKEN COUNT [By Halstead] / Halstead s/w Science measures

- A program is considered to be series of tokens & if we count the no of tokens.
- Tokens are classified as either operators or operands.
- Tokens are combinations of operators & operands
- Measures are functions of the counting these tokens.
- Operators → symbol or keyword, punctuation marks, arithmetic symbols such as +, -, /, *
while, for, printf, special symbol such as =, braces, parentheses, eof.

Operand → symbol used to represent data, variables, constants even labels

→ Size of the vocabulary

$$\eta = \eta_1 + \eta_2$$

- η → vocabulary of a program
- η_1 → no. of unique operators
- η_2 → " " " operands.

→ Length of the program Total no of Tokens used

$$N = N_1 + N_2$$

- N → program length
- N_1 → total occurrences of operators
- N_2 → " " " operands

Volume

$$V = N \times \log_2 \eta$$

unit $\rightarrow$ bits

$\rightarrow$ Volume may also be interpreted as the no. of mental comparisons needed to write a program of length N .

$\rightarrow$ program that is minimal in size is said to have potential volume V^* .

$$L = V^*/V$$

$L \rightarrow$ program level

$\rightarrow$ Range of L is between 0 & 1

$L=1 \rightarrow$ highest possible level i.e. minimum size

Difficulty, $\boxed{D = 1/L}$

As volume of implementation of program increases, program level decreases & difficulty increases.

$$V \uparrow \text{ so } L \downarrow \text{ \& } D \uparrow$$

EFFORT

$\rightarrow$ Effort increase size of program increases.

$$\boxed{E = V/L = D \times V}$$

& it takes more effort to implement a program at lower level (higher difficulty)

Unit:

Elementary Mental Discriminations

Estimated Program length

$$\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 \quad \left. \vphantom{\hat{N}} \right\} \text{Estimated length equation}$$

$\downarrow$ no. of unique operators $\downarrow$ no. of unique operands.

Potential Volume (The volume of the program with minimal size)

$$V^* = (2 + \eta_2^*) \log_2 (2 + \eta_2^*) \quad \left. \vphantom{V^*} \right\} \text{Potential volume of an algorithm implemented as a procedure call}$$

$\eta_2^* \rightarrow$ represent unique o/p & o/p parameters, for small applications only.

$2 \rightarrow$ represents the two unique operators for procedure call.

Estimated program level / Difficulty

$$\hat{L} = 2\eta_2 / (\eta_1 N_2)$$

$$\delta = \frac{1}{\hat{L}} = \frac{\eta_1 N_2}{2\eta_2}$$

Effort

$$E = V / \hat{L} \quad \left[\text{As } \hat{L} = 2\eta_2 / (\eta_1 N_2) \right]$$

$$\therefore E = V \times \frac{\eta_1 N_2}{2\eta_2} \quad \left[\text{As } V = N \log_2 \eta \right]$$

$$\therefore E = \frac{(N \log_2 \eta \eta_1 N_2)}{2\eta_2}$$

Programming Time (T)

$$T = E / \beta$$

$\beta \rightarrow$ speed no. Range $\frac{b/w}{5}$ & 20
by default $\beta \rightarrow 18$.

Operators

- Function call.
- Looping statements, all control statements
e.g. do { } while (),
for () - -
if , if else , switch
- Reserve words →
return, default, continue,
break, sizeof etc
- Bracket, comma & terminators
- GOTO
- ~~For~~ [] operators
e.g. array-name [index]
 ↓ ↓
 operands operator operand
- Dot operator (.)
→ are operators.
- Symbols, keywords, punctuation marks, arithmetic symbol such as +, -, /, *, braces etc.

Operands

- variables, constant
- local variable with same name in different functⁿ are counted as unique operand.
- ~~to~~ GOTO label is operand
- array name [index]
array-name, index
- struct-name → member-name
 ↓ ↓ ↓
 operand operator operand
- symbol used to represent data, variables, constants even labels.

Language Level

$$\boxed{A = L * V^*}$$
$$\quad \quad \quad \downarrow$$
$$\boxed{A = L^2 V}$$

→ It is used to characterize programming language

→ lower value of 1 means it is closer to machine.

$$\boxed{\text{for } C, 1 \rightarrow 0.857}$$

Counting Rules for C language

- ① Comments are not considered.
- ② Identifiers & function declarations are not considered.
- ③ Global variables used in different module of the same program are counted as multiple occurrences of same variable.
- ④ Variables & constant are considered as operands.
- ⑤ Local variable with same name in different function are counted as unique operands.
- ⑥ Function call are considered as operators.
- ⑦ All looping statement e.g. do-while, while, for, if, if-else are counted as operators.
- ⑧ All pre-directives are ignored.

Ques: List out the operators, operators, η , $\sim$, $\vee$, $\wedge$, $\rightarrow$ of foll. program? [Halstead operators].

```
int sort (int x[], int n)
{
    int i, j, save, im1;
    /* This functn sort array */
    if (n < 2) return 1;
    for (i = 2; i <= n; i++)
    {
        im1 = i - 1;
        for (j = 1; j <= im1; j++)
            if (x[i] < x[j])
            {
                save = x[i];
                x[i] = x[j];
                x[j] = save;
            }
    }
    return 0;
}
```


Sol:

OPERATOR	OCCURRENCES	OPERANDS	OCCURRENCES
int	4	sort	1
()	5	x	7
,	4	[]	3
[]	7	n	8
;	11	i.	7
if	2	same	3
<	2	ans	3
returns	2	2	2
for	2	1	3
=	6	0	1
<=	2		
++	2		
-	1		
{ }	3		
$\eta_1 = 14$	$N_1 = 53$	$\eta_2 = 10$	$N_2 = 38$

$$\eta = \eta_1 + \eta_2 = 14 + 10 = 24$$

$$N = N_1 + N_2 = 53 + 38 = 91$$

$$\begin{aligned} \rightarrow V &= N * \log_2 \eta \\ &= 53 * \log_2 24 \\ &= 417 \text{ bits} \end{aligned}$$

$$\begin{aligned} \rightarrow \hat{N} &= \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 \\ &= 14 \log_2 14 + 10 \log_2 10 \\ &= 14 * 3.81 + 10 * 3.32 \\ &= 53.34 + 33.2 = 86.45 \end{aligned}$$

$$\begin{aligned} V^* &= (2 + \eta_2^*) \log_2 (2 + \eta_2^*) \\ &= (2 + 3) * \log_2 (2 + 3) \\ &= 5 * \log_2 5 \\ &= 11.6 \end{aligned}$$

$\eta_2^* \Rightarrow$ unique o/p & o/p parameters
 $\eta_2^* \rightarrow 3$, Array holding the integer to be sorted]

$$\begin{aligned} L &= V^* / V = 11.6 / 417 = 0.027 \\ &= 11.6 / 86.45 = \end{aligned}$$

$$D = 1/L = 1/.027 = 37.03.$$

$$\hat{L} = \frac{2\eta_2}{\eta_1 N_2} = \frac{2 \times 10}{14 \times 38} = .038$$

$$E = \sqrt{\hat{L}} = 417 / .038 = 10973.68. \text{ elementary mental discrimination}$$

$$T = E/\beta = \frac{10973.68}{18} = 610 \text{ seconds} = 10 \text{ minutes}$$

Data Structure Metrics

- It is metrics which captures the amount of data I/p to, processed in and O/p from software.
- A count of the amount of data I/p to, processed in, O/p from S/W is called data structure metric.
- Various data metrics:
 - (a) Concentrate on variables within each module & ignore I/p/O/p dependencies
 - (b) Few concentrate only with I/p/O/p sections.

AMOUNT OF DATA

- This metric proposed to measure the amount of data, the usage of data, within modules & degree to which data is shared among modules.

AMOUNT OF DATA

- It can be measured by generating cross-reference list → this list indicate the line where a certain variable is declared & the line or lines where it is referenced.
- Cross reference list is useful in debugging & maintenance.
- Amount of data is determine by counting no of entries in the cross-reference list.
- VARS → counted variable.
- A variable is a string of alphanumeric characters that is defined by a developer & that is used to represent some value during either compilation or execution.

E.G

```

1 #include <stdio.h>
2 struct check
3 {
4     float gross, tax, net;
5     int pay;
6     float hours, rate;
7 void main()
8 {
9     while (1) {
10         scanf("%f %f", &hours, &rate);
11         pay.gross = hours * rate;
12         pay.tax = .25 * pay.gross;
13         pay.net = pay.gross - pay.tax;
14         printf("%f %f %f\n", pay.gross, pay.tax, pay.net);
15     }
16 }
17 }

```

CROSS REFERENCE LIST

check	2									
gross	4	12	13	14	15					
tax	4	13	14	15						
net	4	14	15							
pay	5	12	13	13	14	14	14	15	15	15
hours	6	11	12							
rate	6	11	12							

$$VARs = 7$$

$$\begin{aligned}
 \eta_2 &= VARs + \text{Unique constants} + \text{labels} \\
 &= 7 + 1 + 0 \\
 &= 8
 \end{aligned}$$

$N_2 \rightarrow$ occurrence of operand

$$\begin{aligned}
 N_2 &= 29 \text{ (count 29 + 1 (unique operand .25 occur once))} \\
 &= 30
 \end{aligned}$$

To calculate amount of data then calculate
VAR, η_2 , N_2

$$VARs = 7$$

$$\text{Unique constant} = 1 \text{ i.e. } .25$$

$$\text{Labels} = 0$$

[Label is function name]

Usage of Data within a Module

Live Variable

- A variable is live from the beginning of a procedure to the end of procedure
- A variable is live at a particular statement only if it is referenced a certain number of statement before or after that statement.
- A variable is live from its first to its last reference within a procedure.

Calculation of Live variable

<u>Line</u>	<u>Live variable</u>	<u>Count</u>
11	hour, rate	2
12	hour, rate, pay, gross, ¹⁵	4
13	pay, gross, tax,	3
14	pay, gross, net, tax	4
15	pay, gross, tax, net	4
16	—	0
17	—	0
		17

$$\text{Average no. of live variable (IV)} = \frac{\text{Sum of count of live variable (IV)}}{\text{Count of executable statement}}$$

$$= \frac{17}{16} = 1.0625$$

Average life of variable (Y)

$$Y = \frac{\text{sum of count of line variable}}{\text{total no. of variable}}$$

$$= \frac{17}{7} = 2.42 \text{ (approx)}$$

Module weakness

$$WM = LV \times Y$$

$$= 1.0625 \times 2.42 = 2.57125$$

→ Weakness of the module can be used to estimate the testability & maintainability

→ If weakness of module is more, testability will be better & vice-versa. ✓

→ ~~If~~ A weaker module will be more difficult to maintain. ✓

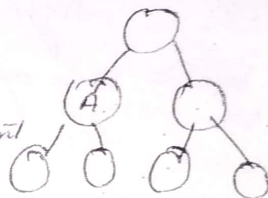
INFORMATION FLOW METRICS (IF)

- Components that are highly coupled & lack cohesion are less reliable & less maintainable than those that are loosely coupled & are cohesive.
- Information flow metrics ^{measure} degree of cohesion & coupling for a particular system component.

Notations used in IF

- Information Flow metrics are applied to the component of a system design; for component 'A' measures are:

- ① FAN IN → count no. of component that can call or pass control to Component A



- ② FAN OUT → No. of components called by component.

- ③ Information flow metric of component A is $IF(A) = [FAN\ IN(A) \times FAN\ OUT(A)]^2$

- ④ LEVEL SUM → Sum of IF value for all components within each level

- ⑤ SYSTEM SUM → Sum the IF value for the total system design is called SYSTEM SUM

→ High FAN IN values indicate component in lack of cohesion

→ ~~FAN OUT~~ High FAN OUT ~~in~~ value also indicate lack of cohesion.

→ High IF value indicate highly coupled component.

→ SYSTEM SUM value is final item of information flow metric.

For component A let,

$$FAN IN(A) = a + b + c + d$$

where

a = no. of components that call A.

b = no. of parameters passed to A from components higher in the hierarchy.

c = no. of parameters passed to A from component lower in hierarchy.

d = no. of data elements read by Component A.

$$FAN IN(A) = a$$

$$FAN OUT(A) = e + f + g + h$$

where

e → no. of components called by A.

f → no. of parameters passed from A to component higher in hierarchy

g → no. of parameters passed from A to component lower in hierarchy

h → no. of data elements written to by A.

More
Sophisticated
IF
Model